

TheHive — Documentación de Entrega

Índice

1. [Descripción del proyecto](#)
 2. [Acceso de demo](#)
 3. [Stack tecnológico](#)
 4. [Arquitectura del proyecto](#)
 - [Arquitectura JAMstack desacoplada](#)
 - [Estructura de directorios](#)
 - [Organización de componentes](#)
 - [Sistema de rutas](#)
 - [Capa de datos — Cliente de Strapi](#)
 - [Sistema de tipos TypeScript](#)
 - [Gestión de estado global](#)
 - [Sistema de carrito](#)
 - [Layout base y configuración dinámica](#)
 - [Despliegue e infraestructura](#)
 5. [Sistema de estilos](#)
 - [Arquitectura del sistema de estilos](#)
 - [Paleta de colores](#)
 - [Sistema de temas — modo claro y oscuro](#)
 - [Tipografía](#)
 - [Transiciones y animaciones](#)
 - [Componentes UI estilizados](#)
 6. [Estrategia SEO](#)
 - [Palabras clave](#)
 - [SEO On-Page](#)
 - [SEO Técnico](#)
 - [Estrategia de contenido](#)
 - [Seguimiento y herramientas](#)
 - [Estado de implementación](#)
 7. [Conclusión](#)
-

Descripción del proyecto

TheHive es una tienda de comercio electrónico orientada a la venta de miel artesanal y productos derivados de la apicultura, desarrollada como Proyecto Integrador del 2.º curso del ciclo formativo de Desarrollo de Aplicaciones Web (DAW).

El sitio implementa una arquitectura desacoplada (JAMstack) con un frontend estático generado en tiempo de compilación y un CMS headless que gestiona todos los contenidos desde un panel de administración, sin necesidad de modificar el código fuente.

Sitio en producción

El proyecto está desplegado y accesible en <https://thehive.pablosuarez.dev>

Acceso de demo

Campo	Valor
URL	https://thehive.pablosuarez.dev
Email	demo@hive.com
Contraseña	proyectopablo

Stack tecnológico

Capa	Tecnología
Frontend	Astro 5 + Tailwind CSS 4
Backend / CMS	Strapi v5 (headless CMS)
Base de datos	PostgreSQL
Servidor web	Nginx — proxy inverso + HTTPS (Let's Encrypt)
Despliegue	VPS en Hetzner
Gestión procesos	PM2
Estado global	Nanostores
Lenguaje	TypeScript

Arquitectura del proyecto

Arquitectura JAMstack desacoplada

TheHive adopta una arquitectura **JAMstack** (JavaScript, APIs, Markup): el frontend y el backend son aplicaciones completamente independientes que se comunican a través de una API REST.

```
[ Navegador ]
  | HTTP
  v
[ Nginx ] -----> [ Frontend Astro ] :4321
  | proxy /api
  v
[ Backend Strapi v5 ] :1337
  |
  v
[ PostgreSQL ]
```

Ventajas de la arquitectura desacoplada

- **Independencia de despliegue:** el frontend se compila y actualiza sin tocar la base de datos.
- **Rendimiento:** Astro genera HTML estático en compilación; el servidor entrega páginas ya renderizadas.
- **Escalabilidad:** una futura app móvil consumiría la misma API sin cambios en el backend.
- **Seguridad:** el token de API de Strapi nunca sale del servidor; el navegador solo accede a `PUBLIC_STRAPI_URL`.

Estructura de directorios

```
TheHive/
├── backend/           # API y CMS – Strapi v5
├── frontend/         # Aplicación web – Astro 5
│   └── src/
│       ├── components/ # Componentes reutilizables
│       └── layouts/    # Layout base compartido
```

```

|   └─ lib/           # Cliente HTTP hacia Strapi
|   └─ pages/         # Rutas del sitio
|   └─ stores/        # Estado global (Nanostores)
|   └─ styles/        # Estilos globales y tokens CSS
|   └─ types/         # Tipos TypeScript
|   └─ utils/         # Utilidades: carrito, helpers
└─ documentation/    # Documentación del proyecto
└─ README.md

```

La separación en `backend/` y `frontend/` refleja la independencia de las dos aplicaciones: cada una tiene su propio `package.json`, servidor y variables de entorno.

Organización de componentes

Los componentes se dividen en cuatro categorías diferenciadas:

`components/ui/` — Bloques genéricos reutilizables

Componente	Función
<code>Button.astro</code>	Botón polimórfico: <code>primary</code> , <code>secondary</code> , <code>outline</code> , <code>ghost</code>
<code>Breadcrumbs.astro</code>	Navegación de migas de pan, estilo homogéneo global
<code>Pagination.astro</code>	Paginación con selector de ítems por página
<code>FilterTabs.astro</code>	Pestañas de filtro para el catálogo
<code>ThemeToggle.astro</code>	Conmutador de tema claro/oscuro
<code>Avatar.astro</code>	Avatar de usuario con menú desplegable
<code>StarRating.astro</code>	Valoración de productos con estrellas
<code>MarkdownRenderer.astro</code>	Renderizado de Markdown proveniente de Strapi

`components/features/` — Componentes por sección

Carpeta	Componentes incluidos
<code>home/</code>	Hero, categorías, productos destacados, testimonios, blog, CTA
<code>product/</code>	Hero del producto, galería, tarjeta, selector de cantidad, reseñas
<code>category/</code>	Hero de categoría, cuadrícula de productos filtrados
<code>blog/</code>	Tarjeta de artículo, cabecera del post, posts relacionados
<code>cart/</code>	Lista de ítems del carrito

Carpeta	Componentes incluidos
contact/	Formulario de contacto y datos de la empresa
faq/	Pregunta y respuesta desplegable

`components/layout/` — `Header.astro` y `Footer.astro`, renderizados en todas las páginas mediante `Layout.astro`.

`components/profile/` — Hero del perfil, tarjeta de pedido y lista de pedidos del área privada.

Por qué organizar por funcionalidad

Agrupar por sección (en lugar de por tipo) localiza bugs más rápido: todos los archivos de `/producto` están en `features/product/`. Añadir una nueva sección no afecta al resto del árbol.

Sistema de rutas

Astro utiliza enrutamiento basado en archivos: cada `.astro` en `src/pages/` se convierte en una ruta.

Archivo	Ruta	Descripción
<code>index.astro</code>	<code>/</code>	Inicio — hero, categorías, productos, blog
<code>productos.astro</code>	<code>/productos</code>	Catálogo con filtros y paginación
<code>categoria/[slug].astro</code>	<code>/categoria/:slug</code>	Productos de una categoría
<code>producto/[slug].astro</code>	<code>/producto/:slug</code>	Ficha de producto individual
<code>blog.astro</code>	<code>/blog</code>	Listado de artículos
<code>blog/[slug].astro</code>	<code>/blog/:slug</code>	Artículo de blog individual
<code>carrito.astro</code>	<code>/carrito</code>	Carrito de compra
<code>checkout.astro</code>	<code>/checkout</code>	Proceso de pago
<code>perfil.astro</code>	<code>/perfil</code>	Perfil e historial de pedidos
<code>login.astro</code> / <code>registro.astro</code>	<code>/login</code> / <code>/registro</code>	Autenticación
<code>sobre-nosotros.astro</code>	<code>/sobre-nosotros</code>	Información de la empresa
<code>contacto.astro</code>	<code>/contacto</code>	Formulario de contacto

Archivo	Ruta	Descripción
faq.astro	/faq	Preguntas frecuentes
404.astro	—	Página de error 404

Capa de datos — Cliente de Strapi

Toda la comunicación con la API se centraliza en `src/lib/strapi.js`, con dos funciones principales:

- `fetchAPI(endpoint, params)` — Petición HTTP con autenticación Bearer; construye la query string recursivamente para soportar filtros anidados de Strapi.
- `get(endpoint, params)` — Llamada simplificada que devuelve `data.data` directamente, eliminando el nivel de anidamiento estándar.

El token de autenticación se obtiene en tiempo de servidor mediante `astro:env/server`, garantizando que nunca se expone al navegador.

```
// Ejemplo de uso en una página Astro
import { get } from "../lib/strapi";

const productos = await get("products", {
  populate: { mainImage: true, category: true },
  sort: { createdAt: "desc" },
});
```

Centralización del cliente

Si la URL de la API o el esquema de autenticación cambian, solo hay que modificar `strapi.js`. Ninguna página ni componente duplica la lógica de autenticación.

Sistema de tipos TypeScript

Todos los modelos de datos están en `src/types/`. Cada fichero corresponde a un recurso de la API:

Fichero	Tipos que define
product.ts	Product , Review , Beekeeper , Origin , Tag
category.ts	Category
blog.ts	BlogPost , BlogCategory
user.ts	User
cart.ts	CartItem
common.ts	StrapiImage , ImageFormat , Seo , Button
home.ts	Tipos de contenido dinámico de la portada
header.ts	Configuración de navegación
footer.ts	Configuración del footer

✓ Beneficio del tipado estricto

Tipar todos los modelos elimina una categoría completa de errores en tiempo de desarrollo. Además, actúa como documentación implícita de la estructura de la API.

Gestión de estado global

El estado compartido entre componentes se gestiona con **Nanostores**, librería ultraligera compatible con Astro, en `src/stores/appStore.ts`.

Store	Tipo	Contenido
\$user	<code>persistentAtom<User \ null></code>	Datos del usuario autenticado y JWT
\$isAuthLoading	<code>atom<boolean></code>	Indica si se está verificando la sesión
\$cart	<code>persistentMap<Record<string, CartItem>></code>	Ítems del carrito

❗ Por qué Nanostores en lugar de Redux o Zustand

Astro no requiere un framework de componentes para la mayor parte de las páginas. Nanostores es framework-agnostic: funciona con Astro y, si una sección futura necesitara componentes React o Svelte, el mismo store se compartiría sin cambios.

Sistema de carrito

Implementado en `src/utils/cartService.ts`, con comportamiento distinto según el tipo de usuario:

Usuario invitado — Los ítems se guardan únicamente en `localStorage` mediante el `persistentMap` de Nanostores. No se realizan peticiones a la API.

Usuario autenticado — Los ítems se persisten en la colección `CartItem` de Strapi. Añadir, modificar cantidad y eliminar se traducen en peticiones `POST`, `PUT` y `DELETE`.

Sincronización tras login — `syncCartAfterLogin` migra los ítems del carrito de invitado a Strapi al iniciar sesión, evitando la pérdida de productos añadidos antes de autenticarse.

Creación de pedido — `createOrder` envía el carrito como un `Order` a Strapi, elimina todos los `CartItem` asociados y vacía el store local.

Compatibilidad con Strapi v5

Strapi v5 usa `documentId` (cadena de texto) en lugar del identificador numérico clásico. Todo el servicio del carrito opera con estas cadenas para mantener compatibilidad con la versión actual del CMS.

Layout base y configuración dinámica

Todas las páginas usan `src/layouts/Layout.astro` como envoltorio. En cada petición, consulta la colección `site-config` de Strapi, que almacena datos editables desde el panel de administración sin necesidad de recompilar el frontend.

El layout gestiona también:

- Detección y aplicación del tema claro/oscuro antes de pintar el HTML (evita FOUC).
- Metaetiquetas SEO: `<title>`, `<meta description>`, Open Graph.

- Etiqueta `<link rel="canonical">` con la URL actual.
- Loader global mientras la página termina de cargar.

Despliegue e infraestructura

El proyecto se despliega en un VPS de Hetzner con la siguiente configuración:

Componente	Herramienta	Detalles
Servidor web	Nginx	Proxy inverso, HTTPS con Let's Encrypt
Gestión de procesos	PM2	Reinicio automático del frontend y backend
Base de datos	PostgreSQL	Gestionada por Strapi
Frontend	Astro <code>npm run build</code>	HTML estático servido por Nginx
Backend	Strapi <code>npm run build</code>	Proceso Node.js en el puerto 1337

```
# Compilar y levantar el frontend
cd frontend && npm run build
pm2 start npm --name "astro" -- run preview

# Levantar el backend
cd backend && npm run build
pm2 start npm --name "strapi" -- run start

# Persistir procesos al reiniciar el servidor
pm2 save && pm2 startup
```

Sistema de estilos

Fuente principal

`frontend/src/styles/global.css` — única fuente de verdad para todos los valores visuales del proyecto.

El sistema se construye sobre una arquitectura de **design tokens** con variables CSS nativas, expuestas como clases utilitarias a través de Tailwind CSS v4. El objetivo es garantizar coherencia visual en todos los componentes y facilitar el mantenimiento.

Arquitectura del sistema de estilos

El sistema combina tres capas que trabajan conjuntamente:

Capa 1 — Tailwind CSS v4: Motor de utilidades

Se importa directamente en `global.css`. Tailwind v4 elimina el fichero `tailwind.config.js` y define tokens directamente en CSS.

```
@import 'tailwindcss';
@plugin '@tailwindcss/typography';
```

Capa 2 — Bloque `@theme {}` : Registro de tokens

Pieza central del sistema. Todos los valores definidos en él quedan registrados y Tailwind genera automáticamente las clases utilitarias correspondientes.

```
@theme {
  --color-primary: #E89B00;
  --color-bg-secondary: #FAF7F2;
  --color-text-primary: #1A1A1A;
}
```

Generación automática de clases

Definir `--color-primary` genera `bg-primary`, `text-primary`, `border-primary`, etc. sin configuración adicional.

Capa 3 — Variables en `:root` : Capa semántica

Los mismos tokens se reexponen como alias cortos para uso directo en bloques `<style>` de componentes Astro.

```
:root {
  --primary: var(--color-primary);
  --bg-secondary: var(--color-bg-secondary);
  --text-primary: var(--color-text-primary);
}
```

Paleta de colores

Colores de marca — Tonos cálidos de miel y madera.

Token CSS	Clase	Valor	Uso
--color-primary	*-primary	#E89B00	Dorado miel. Color principal.
--color-primary-dark	*-primary-dark	#C27A00	Variante oscura. Estados hover.
--color-primary-light	*-primary-light	#F6C65B	Variante clara. Fondos suaves.
--color-primary-pale	*-primary-pale	#FFF3D6	Tono tenue. Fondo outline.
--color-secondary	*-secondary	#5C2E0C	Marrón oscuro. Secundario.
--color-secondary-light	*-secondary-light	#7A3E12	Variante clara del secundario.
--color-accent	*-accent	#F2B705	Acento complementario.

Colores de fondo — Jerarquía visual de secciones (modo claro).

Token CSS	Valor	Uso
--color-bg-primary	#FFFFFF	Fondo base de la página.
--color-bg-secondary	#FAF7F2	Fondo de secciones y tarjetas.
--color-bg-tertiary	#F3EEE6	Fondo de elementos anidados.
--color-bg-footer	#FFF3D6	Fondo específico del footer.

Colores de texto — Tres niveles de jerarquía.

Token CSS	Valor	Uso
--color-text-primary	#1A1A1A	Texto principal y titulares.
--color-text-secondary	#5B5B5B	Subtítulos y metadatos.
--color-text-tertiary	#8A8A8A	Pies de página y marcas de tiempo.
--color-text-inverse	#FFFFFF	Texto sobre fondos oscuros.

Colores de borde y estado semántico.

Token CSS	Valor	Uso
--color-border-default	#E8E2D8	Borde estándar de inputs y tarjetas.

Token CSS	Valor	Uso
<code>--color-divider</code>	<code>#DDD6C8</code>	Líneas divisorias horizontales.
<code>--color-success</code>	<code>#15803D</code>	Operaciones completadas con éxito.
<code>--color-error</code>	<code>#DC2626</code>	Fallos y validaciones incorrectas.
<code>--color-warning</code>	<code>#D97706</code>	Avisos de alerta moderada.
<code>--color-info</code>	<code>#2563EB</code>	Mensajes informativos neutros.

Sistema de temas — modo claro y oscuro

El cambio de tema opera con tres capas de prioridad:

1. **Detección automática** — `prefers-color-scheme` aplica el tema del SO si no hay preferencia guardada.
2. **Preferencia manual** — El atributo `data-theme` en `<html>` (`"light"` / `"dark"`) sobrescribe la detección automática. Se activa con `ThemeToggle.astro`.
3. **Persistencia** — La preferencia se guarda en `localStorage` para evitar el parpadeo de tema (FOUC) al cargar la página.

Valores de tokens en modo oscuro — Fondos en escala de grises azulados; primario aclarado para mantener contraste.

Token	Modo claro	Modo oscuro
<code>--color-bg-primary</code>	<code>#FFFFFF</code>	<code>#111827</code>
<code>--color-bg-secondary</code>	<code>#FAF7F2</code>	<code>#1F2937</code>
<code>--color-bg-tertiary</code>	<code>#F3EEE6</code>	<code>#374151</code>
<code>--color-text-primary</code>	<code>#1A1A1A</code>	<code>#F9FAFB</code>
<code>--color-text-secondary</code>	<code>#5B5B5B</code>	<code>#D1D5DB</code>
<code>--color-text-tertiary</code>	<code>#8A8A8A</code>	<code>#9CA3AF</code>
<code>--color-border-default</code>	<code>#E8E2D8</code>	<code>#374151</code>
<code>--color-primary</code>	<code>#E89B00</code>	<code>#FBBF24</code>
<code>--color-primary-dark</code>	<code>#C27A00</code>	<code>#F59E0B</code>
<code>--color-primary-light</code>	<code>#F6C65B</code>	<code>#FCD34D</code>
<code>--color-primary-pale</code>	<code>#FFF3D6</code>	<code>#78350F</code>

Tipografía

Todos los tamaños son **fluidos**: se calculan con `clamp(mínimo, preferido, máximo)`, escalando con el ancho de ventana sin media queries.

```
h1 {
  font-size: clamp(2rem, 5vw, 3.5rem);
  margin-bottom: clamp(1rem, 3vw, 1.5rem);
}
```

Elemento	Mín.	Dinámico	Máx.	Margen inf.
h1	2rem	5vw	3.5rem	1rem - 1.5rem
h2	1.5rem	4vw	2.5rem	0.75rem - 1.25rem
h3	1.25rem	3vw	2rem	0.5rem - 1rem
h4	1.125rem	2.5vw	1.5rem	0.5rem - 0.875rem
h5	1rem	2vw	1.25rem	0.5rem - 0.75rem
h6	0.875rem	1.5vw	1.125rem	0.5rem - 0.625rem
p	1rem	1.5vw	1.125rem	0.75rem - 1rem

Los elementos `<a>` no tienen subrayado por defecto y cambian a `--primary-light` en hover con `transition: 0.2s ease`.

Transiciones y animaciones

Tres velocidades estándar definidas como variables CSS globales:

Variable	Valor	Propósito
<code>--transition-fast</code>	<code>150ms ease-in-out</code>	Feedback inmediato: cambios de icono, clics.
<code>--transition-base</code>	<code>250ms ease-in-out</code>	Estándar: cambios de color, fondo, borde.
<code>--transition-slow</code>	<code>350ms ease-in-out</code>	Animaciones amplias: apertura de menús/paneles.

Componentes UI estilizados

Button.astro

Ruta: frontend/src/components/ui/Button.astro

Componente polimórfico: renderiza `<a>` si recibe `link` , o `<button>` si recibe `type` . Cuatro variantes jerarquizadas:

Variante	Fondo	Texto	Hover
primary	<code>--color-primary</code>	<code>--text-inverse</code>	<code>--color-primary-dark</code>
secondary	<code>--color-secondary</code>	<code>--text-inverse</code>	<code>--color-secondary-light</code>
outline	Transparente	<code>--color-primary</code>	<code>--color-primary-pale</code>
ghost	Transparente	<code>--color-primary</code>	<code>--color-primary-dark</code>

Clases base comunes a todas las variantes:

```
inline-flex items-center justify-center rounded-lg font-semibold transition-all duration-200 cursor-pointer
```

Breadcrumbs.astro

Navegación secundaria dinámica basada en `Astro.url.pathname` . El segmento activo se renderiza como `<span aria-current="page">` en lugar de enlace.

Prop	Tipo	Por defecto	Descripción
<code>homeLabel</code>	<code>string</code>	<code>"Home"</code>	Etiqueta del primer segmento.
<code>separator</code>	<code>string</code>	<code>">"</code>	Carácter separador.

ThemeToggle.astro

Conmutador de tema claro/oscuro. Contiene un checkbox accesible oculto y dos SVG (sol/luna) que se intercambian según el tema activo. Gestiona `localStorage` y aplica el atributo `data-theme` al elemento `<html>` .

Estrategia SEO

TheHive integra una estrategia de posicionamiento orgánico que cubre aspectos técnicos y de contenido, con el objetivo de atraer tráfico cualificado sin recurrir a publicidad de pago.

Palabras clave

Se aplica un enfoque de **cola larga** (*long-tail keywords*): consultas específicas con menor competencia y mayor intención de compra.

Página	Keyword principal	Propósito
Inicio /	"tienda miel online España"	Tráfico general de marca
Catálogo /productos	"miel ecológica online"	Intención de compra
Categoría /categoria/...	"[tipo de miel] comprar"	Segmentación por producto
Producto /producto/...	"[nombre del producto] precio"	Conversión directa
Blog /blog	Variadas según artículo	Tráfico informativo
Sobre nosotros	"apicultores artesanales"	Autoridad y confianza

SEO On-Page

Metaetiquetas — título y descripción

Gestionadas desde `Layout.astro`. El `<title>` sigue el formato `[Keyword] | TheHive`; la descripción se limita a 160 caracteres.

```
<Layout
  title="Miel Ecológica Online"
  description="Compra miel ecológica y artesanal directamente del apicultor.
Envío a toda España."
>
```

Estructura semántica

- **Jerarquía de encabezados:** Cada página tiene un único `<h1>` con la keyword principal; `<h2>` y `<h3>` estructuran el contenido.
- **URLs semánticas:** Legibles, en minúsculas, gestionadas desde Strapi.

Patrón evitado	Patrón adoptado
/producto?id=47	/productos/miel-de-romero-500g
/cat/1/subcategoria	/categoria/mieles-monofloral

SEO Técnico

Sitemap y Robots

El paquete `@astrojs/sitemap` genera automáticamente `sitemap-index.xml` en cada compilación. El archivo `robots.txt` excluye las páginas privadas:

```
User-agent: *
Allow: /
Disallow: /perfil
Disallow: /carrito
Disallow: /checkout

Sitemap: https://thehive.es/sitemap-index.xml
```

Datos estructurados (Schema.org)

Marcado **JSON-LD** en fichas de producto para habilitar *rich snippets* (precio y disponibilidad) en los resultados de búsqueda de Google.

Rendimiento y Core Web Vitals

Google utiliza estas métricas como señal de posicionamiento:

Métrica	Descripción	Umbral óptimo
LCP	Tiempo de carga del elemento principal	< 2,5 s
INP	Tiempo de respuesta a la interacción	< 200 ms
CLS	Estabilidad visual (movimiento de layout)	< 0,1

Análisis en PageSpeed Insights:



Móvil



Ordenador

Diagnostica problemas de rendimiento

100

Rendimiento

90

Accesibilidad

100

Prácticas
recomendadas

100

SEO

100

Rendimiento

Los valores son estimaciones y pueden variar. La [puntuación del rendimiento se calcula](#) directamente a partir de estas métricas. [Ver calculadora](#).



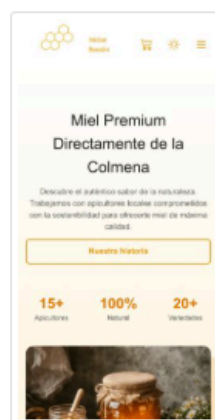
0-49



50-89



90-100



MÉTRICAS

[Ampliar vista](#)

Enlace del análisis: https://pagespeed.web.dev/analysis/https-thehive-pablosuarez-dev/9zyu7qi8hh?form_factor=mobile

Estrategia de contenido

El blog como motor de tráfico

Cada artículo publicado es una nueva URL indexable. Consultas informativas como "beneficios de la miel de tomillo" sirven de puerta de entrada hacia el catálogo comercial.

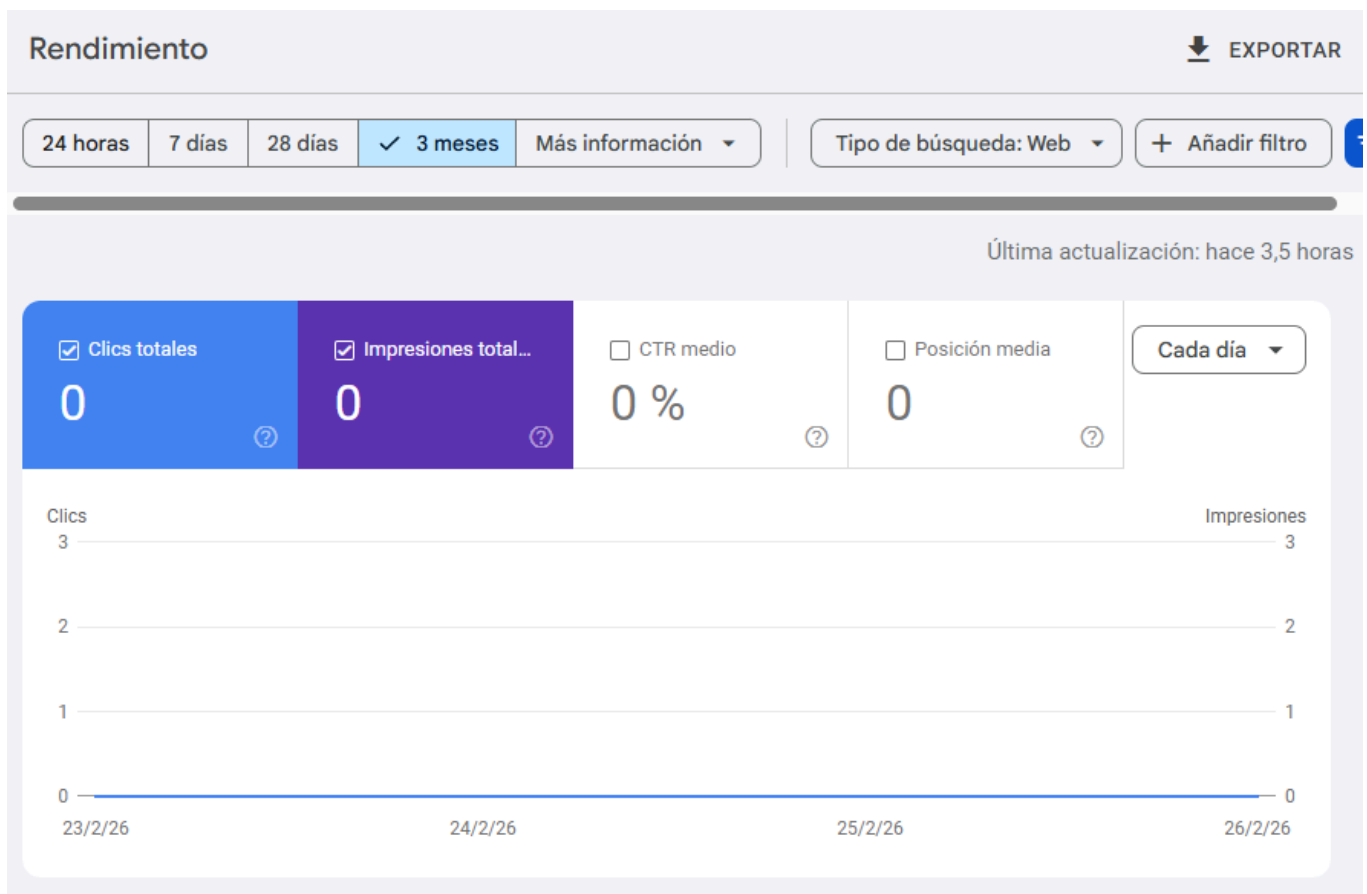
- **Descripciones únicas:** No se reutiliza texto de proveedores para evitar penalizaciones por contenido duplicado.

- **Open Graph:** Metaetiquetas en `Layout.astro` para controlar la previsualización al compartir en redes sociales.

Seguimiento y herramientas

Herramienta	Función
Google Search Console	Monitorización de indexación y errores de rastreo
PageSpeed Insights	Medición periódica de Core Web Vitals

Panel de Google Search Console (el sitemap fue enviado recientemente, aún sin datos de rendimiento):



Confirmación de envío del `sitemap.xml` :

Sitemaps enviados						
Sitemap	Tipo	Enviado ↓	Última lectura	Estado	Páginas descubiertas	Vídeos descubiertos
/sitemap.xml	Sitemap	25 feb 2026	1 mar 2026	Correcto	26	0

 Tiempo de espera

Se recomienda entre 1 y 2 semanas desde el envío del sitemap para comenzar a recibir métricas en Search Console.

Estado de implementación

Elemento	Estado
Títulos y descripciones únicos	Implementado
Etiquetas Open Graph	Implementado
Sitemap XML generado y enviado	Implementado
Archivo robots.txt configurado	Implementado
Certificado SSL (HTTPS) activo	Implementado
Rendimiento > 80 en PageSpeed	Implementado
Atributos alt en todas las imágenes	Implementado
Datos estructurados Schema.org	Pendiente
Google Business Profile	Pendiente

Conclusión

TheHive es un proyecto de comercio electrónico completo que integra un frontend moderno (Astro 5) con un CMS headless (Strapi v5), desplegado en producción sobre un VPS con Nginx y PM2. La arquitectura JAMstack desacoplada permite que cada capa evolucione de forma

independiente, mantiene la seguridad de las claves de API y ofrece un rendimiento óptimo gracias al renderizado estático en tiempo de compilación.

El sistema de estilos basado en design tokens garantiza coherencia visual global y soporte nativo de modo oscuro sin modificaciones individuales por componente. La estrategia SEO cubre los aspectos técnicos fundamentales para el posicionamiento orgánico, con el sitemap ya enviado a Google Search Console y a la espera de los primeros datos de rendimiento.